

Things I've been building, lately

I've been having a lot of fun exploring projects lately. Below are just a handful of projects I've been messing with. Some are utilities. Some are learning projects. All of them

without ever restarting a process.

```
1 use beagle.socket as socket
2
3 fn main() {
4     let listener = socket/listen("0.0.0.0", 8080)
5     println("Echo server listening on port 8080")
6
7     socket/on-connection(listener, fn(conn) {
8         loop {
9             let data = socket/read(conn, 4096) // fully a
```

```
15     }
16   }
17 }
18 }
```

FEATURE RICH LANGUAGE

01	Algebraic effects Built-in algebraic effects make it easy to control and test IO	02	Live coding Redefine functions, structs, and enums while your code is running
03	Async without function coloring Delimited continuations give us the power to have transparent asynchronous	04	Compiled No VM, no bytecode – direct machine code

// keep-running

Detach, but with a friendlier CLI and a replay buffer

```
$ keep-running run -n build -- cargo build --release
→ session 'build' started - pid 84219
→ detach with ctrl-\ ctrl-\

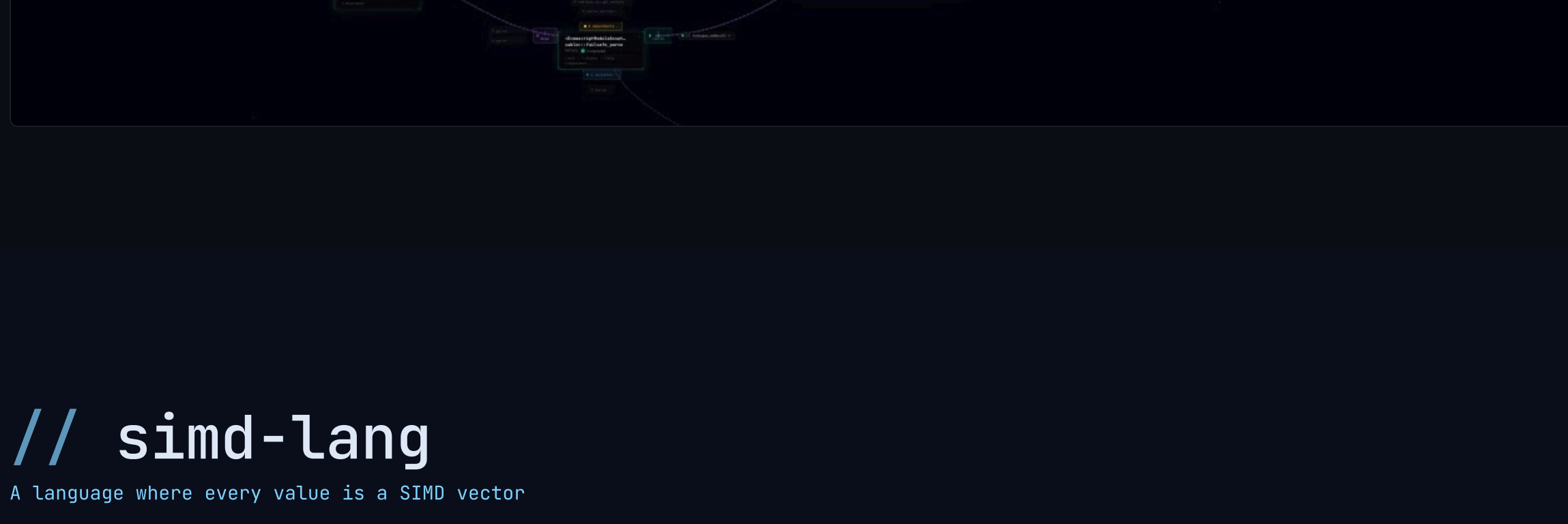
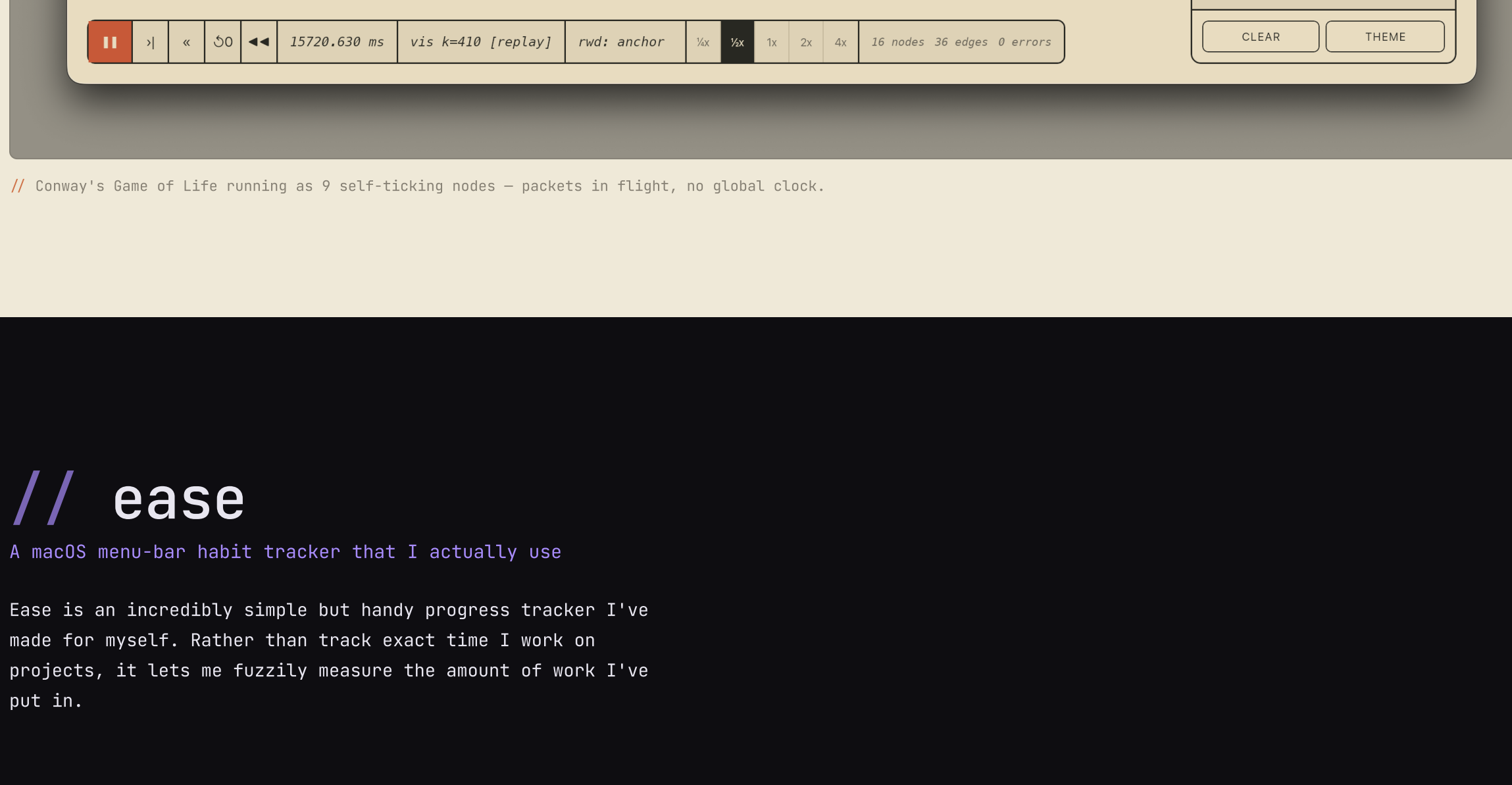
$ keep-running ls
```

NAME	PID	STATUS	UPTIME	BUF
build	84219	running	14m	482 KB
tests	84402	running	3m	11 KB

1. **Introduction**
 2. **Background**
 3. **Methodology**
 4. **Results**
 5. **Conclusion**
 6. **References**
 7. **Appendix**
 8. **Index**
 9. **Table of Contents**
 10. **Summary**
 11. **Abstract**
 12. **Keywords**
 13. **Subject**
 14. **Topic**
 15. **Field**
 16. **Area**
 17. **Discipline**
 18. **Branch**
 19. **Department**
 20. **Faculty**
 21. **School**
 22. **College**
 23. **University**
 24. **Institution**
 25. **Organization**
 26. **Company**
 27. **Enterprise**
 28. **Business**
 29. **Industry**
 30. **Market**
 31. **Sector**
 32. **Field**
 33. **Area**
 34. **Discipline**
 35. **Branch**
 36. **Department**
 37. **Faculty**
 38. **School**
 39. **College**
 40. **University**
 41. **Institution**
 42. **Organization**
 43. **Company**
 44. **Enterprise**
 45. **Business**
 46. **Industry**
 47. **Market**
 48. **Sector**
 49. **Field**
 50. **Area**
 51. **Discipline**
 52. **Branch**
 53. **Department**
 54. **Faculty**
 55. **School**
 56. **College**
 57. **University**
 58. **Institution**
 59. **Organization**
 60. **Company**
 61. **Enterprise**
 62. **Business**
 63. **Industry**
 64. **Market**
 65. **Sector**
 66. **Field**
 67. **Area**
 68. **Discipline**
 69. **Branch**
 70. **Department**
 71. **Faculty**
 72. **School**
 73. **College**
 74. **University**
 75. **Institution**
 76. **Organization**
 77. **Company**
 78. **Enterprise**
 79. **Business**
 80. **Industry**
 81. **Market**
 82. **Sector**
 83. **Field**
 84. **Area**
 85. **Discipline**
 86. **Branch**
 87. **Department**
 88. **Faculty**
 89. **School**
 90. **College**
 91. **University**
 92. **Institution**
 93. **Organization**
 94. **Company**
 95. **Enterprise**
 96. **Business**
 97. **Industry**
 98. **Market**
 99. **Sector**
 100. **Field**
 101. **Area**
 102. **Discipline**
 103. **Branch**
 104. **Department**
 105. **Faculty**
 106. **School**
 107. **College**
 108. **University**
 109. **Institution**
 110. **Organization**
 111. **Company**
 112. **Enterprise**
 113. **Business**
 114. **Industry**
 115. **Market**
 116. **Sector**
 117. **Field**
 118. **Area**
 119. **Discipline**
 120. **Branch**
 121. **Department**
 122. **Faculty**
 123. **School**
 124. **College**
 125. **University**
 126. **Institution**
 127. **Organization**
 128. **Company**
 129. **Enterprise**
 130. **Business**
 131. **Industry**
 132. **Market**
 133. **Sector**
 134. **Field**
 135. **Area**
 136. **Discipline**
 137. **Branch**
 138. **Department**
 139. **Faculty**
 140. **School**
 141. **College**
 142. **University**
 143. **Institution**
 144. **Organization**
 145. **Company**
 146. **Enterprise**
 147. **Business**
 148. **Industry**
 149. **Market**
 150. **Sector**
 151. **Field**
 152. **Area**
 153. **Discipline**
 154. **Branch**
 155. **Department**
 156. **Faculty**
 157. **School**
 158. **College**
 159. **University**
 160. **Institution**
 161. **Organization**
 162. **Company**
 163. **Enterprise**
 164. **Business**
 165. **Industry**
 166. **Market**
 167. **Sector**
 168. **Field**
 169. **Area**
 170. **Discipline**
 171. **Branch**
 172. **Department**
 173. **Faculty**
 174. **School**
 175. **College**
 176. **University**
 177. **Institution**
 178. **Organization**
 179. **Company**
 180. **Enterprise**
 181. **Business**
 182. **Industry**
 183. **Market**
 184. **Sector**
 185. **Field**
 186. **Area**
 187. **Discipline**
 188. **Branch**
 189. **Department**
 190. **Faculty**
 191. **School**
 192. **College**
 193. **University**
 194. **Institution**
 195. **Organization**
 196. **Company**
 197. **Enterprise**
 198. **Business**
 199. **Industry**
 200. **Market**
 201. **Sector**
 202. **Field**
 203. **Area**
 204. **Discipline**
 205. **Branch**
 206. **Department**
 207. **Faculty**
 208. **School**
 209. **College**
 210. **University**
 211. **Institution**
 212. **Organization**
 213. **Company**
 214. **Enterprise**
 215. **Business**
 216. **Industry**
 217. **Market**
 218. **Sector**
 219. **Field**
 220. **Area**
 221. **Discipline**
 222. **Branch**
 223. **Department**
 224. **Faculty**
 225. **School**
 226. **College**
 227. **University**
 228. **Institution**
 229. **Organization**
 230. **Company**
 231. **Enterprise**
 232. **Business**
 233. **Industry**
 234. **Market**
 235. **Sector**
 236. **Field**
 237. **Area**
 238. **Discipline**
 239. **Branch**
 240. **Department**
 241. **Faculty**
 242. **School**
 243. **College**
 244. **University**
 245. **Institution**
 246. **Organization**
 247. **Company**
 248. **Enterprise**
 249. **Business**
 250. **Industry**
 251. **Market**
 252. **Sector**
 253. **Field**
 254. **Area**
 255. **Discipline**
 256. **Branch**
 257. **Department**
 258. **Faculty**
 259. **School**
 260. **College**
 261. **University**
 262.

Test Systems.

© 2006 The Authors
Journal compilation © 2006 Blackwell Publishing Ltd



```
1 fn json_stage1(input: ptr[u8], positions:
2     stream chunk: u8[64] over input carry
```

```

4      bs_vec      = chunk == '"'
5      bs_vec      = chunk == '\\'
6      real_quote_vec = quote_vec & ^ lane_shr(bs_vec, 1)
7
8      structural_vec = (chunk == '{') | (chunk == '}')
9                      | (chunk == '[') | (chunk == ']')
10
11      all_ones: u64[1] = ~0
12      quote_bits      = to_bitmask(real_quote_vec)
13      in_string_raw    = c1mul(quote_bits, all_ones)
14      in_string_with_carry = in_string_raw ^ prev_in_st
15

```

```

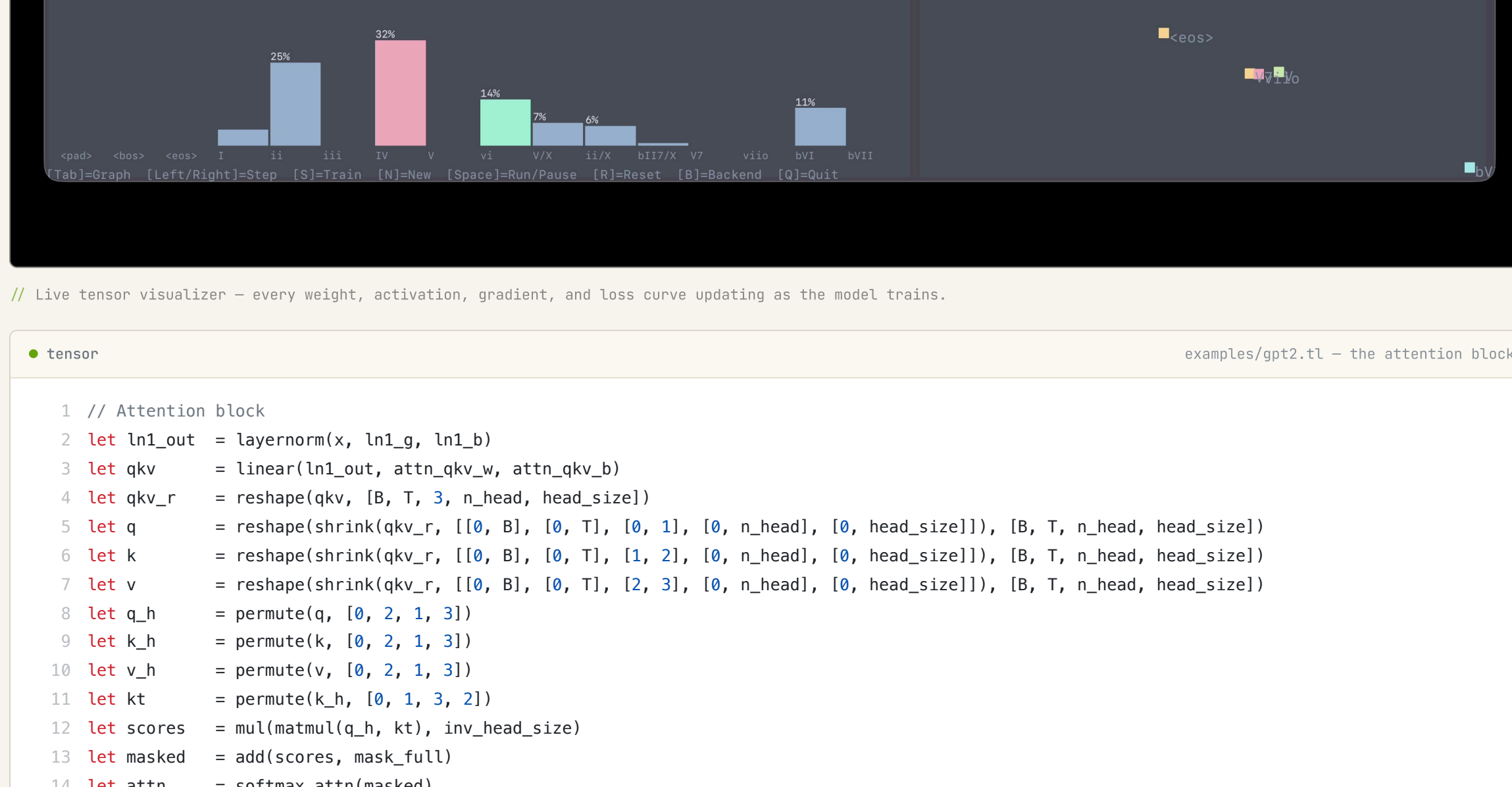
17     real_structural_vec = from_bitmask(real_structural_vec,
18                                         indices, chunk_offset)
19     indices = iota(64) + chunk_offset
20     compressstorepositions, pos, indices, real_structural_vec)
21
22     carry prev_in_string = (in_string_with_carry >> 6)
23     carry pos = pos + popcount(real_structural_vec)
24 }
25
26 return pos

```

```
// tensor-lang
```

I then take this and compile it into ARM mac (scalar), WASH (SIMD), and WGPU. As part of tiny model on some jazz chord progressions a where you can watch it train live and inspec

All of this is powered by my compilers and DSL.



```
15 let attn_out = matmul(attn, v_h)
16 let merged = reshape/permute(attn_out, [0, 2, 1, 3]) @ [B, T, n_embd]
```

baseline (old ARM backend)	3740 ms	1.08x
• nested loops	3595 ms	1.04x
• k-invariant hoisting	3201 ms	1.17x
• matmul unfusion • MR=8 kernel	1475 ms	2.54x
• pointer incrementing	1434 ms	2.61x
• all combined	198 ms	18.98x
lln.c -O3 (reference)	84 ms	—
// GPT-2 124M • 12 layers • T=16 • single-threaded • Apple Silicon		

// dataLog-db
A Datomic-shaped database with a real source

I've grown to love datalog. It is a fascinating way to

This is only a POC right now, but the goal is to build a proper Datalog database on top of a key-value store. But I don't want users to need to think about the exact details of the underlying store and instead work in a much higher-level fashion. To that end, I've added something I always want in a DB: algebraic data types.

```

11
12 assert User { name: "Alice", age: 30, email: "alice@example.com" }
13 assert User { name: "Bob", age: 25, email: "bob@example.com" }
14 assert User { name: "Charlie", age: 35, email: "charlie@example.com" }
15
16 find ?name, ?age
17   where ?u: User { name: ?name, age: > 27 }
18   as of time "2025-04-20T00:00:00Z"

```